Retail Run: Autonomous Navigation and Dynamic Obstacles in a Smart Store

Soham Mulik and Rudra Tere
Mentor: Prof. Pallavi Nikumbh Dhade
Department of Computer Science and Engineering (AI & ML)
Pimpri Chinchwad College of Engineering, Pune

***Abstract*—This paper presents Retail Run, a browser simulation in which a robot completes store tasks and responds to changing aisle conditions. Five search methods are compared on a shared grid. A rule-based controller selects orders, replenishment, and charging without repeated manual commands. Temporary obstacles trigger route updates. Across six normal-cost queries, A* used 81 expansions compared with 754 for breadth-first search, with equal total route cost. In two deterministic 120-second autonomous runs, both conditions completed 11 missions; enabling obstacles increased movement cost from 289 to 299. These observations demonstrate the implemented behavior on one store layout rather than general performance in physical retail environments.**

***Index Terms*—Artificial intelligence, smart retail, path planning, A*, autonomous agents, dynamic obstacles.**

I. INTRODUCTION

A store robot must do more than reach a shelf. It must select a task, travel through available aisles, carry products, and continue when a planned route becomes blocked. Retail Run turns these activities into an interactive environment in which the user can observe each decision and alter the conditions during execution.

The project develops the smart-retail scenario proposed in our FA1 study [3]. Its game-style navigation is inspired by Rescue Run [4], while the goals are retail operations: collecting products, completing deliveries, replenishing shelves, guiding customers, inspecting departments, and charging.

The study asks two practical questions: how do elementary search strategies behave on the same store, and can a simple autonomous controller continue useful work when temporary obstacles appear? The contribution is an inspectable application and a reproducible set of small experiments, not a new search algorithm.

II. STORE AND NAVIGATION PROBLEM

A. Environment

The application represents a store as a 24 by 15 grid. Shelves occupy blocked cells, and a robot moves between adjacent cells in four directions. Six departments contain produce, bakery items, dairy products, pantry goods, household supplies, and drinks. A dock, stockroom, pickup area, and entrance provide distinct mission destinations.

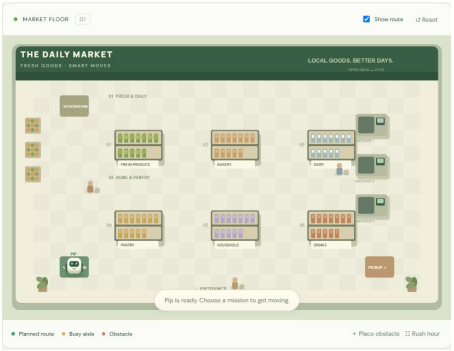


Fig. 1. Retail Run store interface. Shelves define the aisles, while the robot follows a route between task checkpoints.

The navigation state is the current cell. The complete application state also records the mission stage, battery, carried load, shelf stock, pending orders, and active obstacles. For route planning, a path is an ordered sequence of traversable cells:

$$P = (v_0, v_1, \ldots, v_k), \; C(P) = \Sigma\, c(v_i, v_{i+1}).$$

Ordinary movement costs one unit. Rush-hour mode raises the cost to four units in the central band, at columns 9 through 14. The robot cannot enter shelves or active obstacle cells. The simulator exposes the complete grid to the planner; it does not estimate obstacles from sensors.

B. Tasks and resources

A delivery contains collection and pickup stages. Restocking first visits the stockroom and then the relevant shelf. Products weigh one kilogram per unit, and the robot carries at most eight kilograms. Each department initially has 12 units on its shelf and 30 in reserve. Movement consumes 0.18 battery percentage points per cost unit.

III. METHODS

A. Search strategies

Breadth-first search (BFS) expands a FIFO frontier and finds a shortest path in number of moves when edge costs are equal. Depth-first search (DFS) follows a branch before backtracking; its first route need not be short. Uniform-cost search (UCS) selects the smallest accumulated cost g(n). Greedy best-first search selects the smallest estimated remaining distance h(n) [1].

A* combines the travelled and estimated remaining costs [2]:

$$f(n) = g(n) + h(n),$$
$$h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|.$$

The Manhattan estimate fits four-direction movement. Every allowed move costs at least one, so it does not overestimate the remaining route cost in either cost mode. Priority frontiers are sorted arrays, with lexical cell-key tie breaking. The implementation checks for the goal when selecting a node and avoids redundant exploration. Expansion counts therefore describe this implementation, including its tie order.

B. Autonomous task selection

Autonomy uses explicit priorities rather than learning a policy. When the robot is idle, a battery level below 35% selects charging. Otherwise, a shelf at four units or fewer is replenished if reserve stock remains. Pending customer orders are served when neither higher-priority condition applies. A running mission continues through its own stages.

Incoming orders are generated every 12 event seconds, with at most six waiting orders. Department selection cycles through the store. This bounded, deterministic arrival process makes the demonstration repeatable, but it does not represent measured customer demand.

Listing 1. Simplified controller logic; route execution occurs between decisions.

```
if idle:
    if battery < 35: charge()
    elif low_stock_available(): restock()
    elif orders_waiting(): serve_order()
if route_invalid: replan()
if no_route: wait_for_change()
```

The priorities separate selecting a destination from reaching it. Search determines a route for the current stage; the controller determines which mission should start next.

C. Dynamic obstacles and recovery

The environment can introduce temporary trolleys, spills, or crates. The first automatic event occurs after six seconds; later events occur every 14 seconds, and each blockage lasts 18 seconds. A future route cell is selected when possible so that the disturbance affects the current mission. Users may also add blockages manually.

When the remaining route becomes invalid, the planner runs again from the current position. The interface briefly retains the previous path in coral and displays the replacement in green. If the target is unreachable, the robot waits safely and attempts recovery after the environment changes. Pausing freezes both movement and event timers.

Observe mission and aisle state
↓
Choose task when idle
↓
Plan and execute next stage
↓
Replan after a blockage or wait

Fig. 2. Control sequence for autonomous execution. The state is observed again throughout the mission.

D. Scope of the AI workbench

Two additional demonstrations connect the application to the FA1 topics. A small constraint-satisfaction example assigns three loads to robots subject to capacity, battery, and unique-assignment constraints. Backtracking searches the possible assignments. A separate Minimax example compares staff assistance, rerouting, and continuing under fixed outcome scores; alpha-beta pruning skips branches that cannot change the decision.

These examples are teaching panels. They do not control a fleet, reserve shared aisles, or coordinate the moving store robot. The measured navigation and autonomous results use the single-robot controller described above.

E. Evaluation procedure

The evaluation uses the actual JavaScript application through a headless test harness. Six dock-to-department routes are evaluated for each algorithm with normal and rush-hour costs. Results aggregate movement cost and expanded nodes over the six queries. A separate intervention blocks a known route cell.

Two autonomous trials advance the simulator through 1,200 steps of 100 milliseconds, giving 120 seconds at normal speed. Both start with two-unit apple and milk orders. One enables temporary obstacles and the other disables them. The layout, initial resources, arrival sequence, and controller remain the same. Measurements were recorded on 27 October 2026 using Node.js v24.22.1 [5].

IV. RESULTS

A. Search comparison

Table I summarizes the six normal-cost queries. A* and Greedy expanded 81 nodes in total, compared with 754 for BFS and 737 for UCS. All four returned total cost 81. DFS returned a much longer set of routes, with cost 927. The equality among the other methods is specific to these start-goal pairs and aisle geometry.

TABLE I. NORMAL-COST QUERIES: SIX-ROUTE TOTALS

Method	Route cost	Expanded
BFS	81	754
DFS	927	947
UCS	81	737
Greedy	81	81
A*	81	81

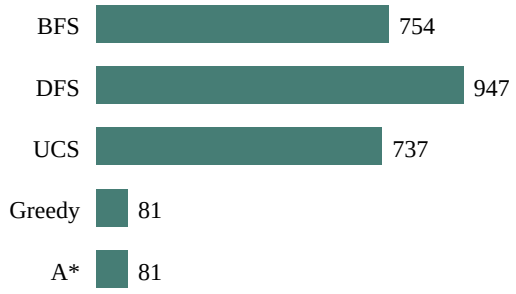


Fig. 3. Expanded nodes under normal movement costs. Values are totals across six fixed queries.

Rush-hour movement changes the cost landscape without changing the shelf layout. BFS, UCS, Greedy, and A* all returned total cost 141 in these queries; DFS returned 1,557. A* expanded 504 nodes, more than its 81 under normal costs. Its distance heuristic remains valid but describes the weighted routes less closely.

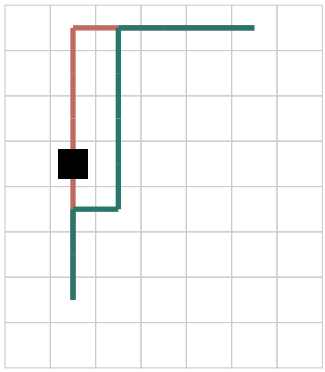
TABLE II. RUSH-HOUR QUERIES: SIX-ROUTE TOTALS

Method	Route cost	Expanded
BFS	141	754
DFS	1557	947
UCS	141	775
Greedy	141	81
A*	141	504

An expansion count is not an elapsed-time measurement. Sorting an array frontier has its own cost, and the present experiment does not benchmark wall-clock speed. Likewise, matching UCS here does not make BFS or Greedy generally cost-optimal on weighted maps.

B. Controlled route change

A produce mission begins at dock cell (2,12) and targets (6,6). Its initial route includes cell (2,9). Blocking that cell caused one replanning event. The robot shifted into the adjacent aisle and still reached the same target with route cost 10. An environmental change can therefore alter the route without necessarily increasing its cost.



Coral: original Green: revised Black: blockage

Fig. 4. Local view of the controlled blockage at (2,9). Both paths contain ten moves; the replacement avoids the blocked cell.

C. Autonomous execution

Both 120-second trials completed 11 missions and ended with no queued orders or active mission. With temporary obstacles enabled, movement cost rose by 10 units, from 289 to 299, an increase of approximately 3.46%. Fifteen replanning events were recorded. The same number of completions does not imply identical routes or resource use.

TABLE III. AUTONOMOUS TRIALS OVER 120 SECONDS

Metric	No events	Events
Completed missions	11	11
Movement cost	289	299
Replans	0	15
Final battery (%)	47.98	46.18
Pending orders	0	0

The 1.80 percentage-point battery difference follows directly from the ten extra cost units and the consumption rule. Neither trial reached the charging threshold or required replenishment. Those branches were checked separately by setting battery to 25% and shelf stock to three units. The controller then selected charging and stock collection, respectively.

D. Reading the application output

The displayed route is a plan for the current mission stage. Its cost reflects terrain weights, while the explored cells show the search effort required to obtain it. A longer-looking exploration trace does not necessarily mean the executed route is longer. The old and revised path overlays make this difference visible during a blockage demonstration.

For a repeatable classroom demonstration, start from a reset state, select a search method, and dispatch the same department task. Record its cost and expansions before changing the method. To demonstrate environmental change, add a blockage along the planned route or enable temporary events. The relevant evidence is whether the robot avoids the blocked cell, replans, and completes or safely waits.

The autonomous queue and completed-mission counter answer a different question: whether the robot continues serving work without repeated commands. They should be interpreted together with movement cost and remaining battery. A completion count alone hides the extra travel introduced by obstacles.

V. DISCUSSION

The results show how task priorities and graph search can be combined in a compact retail simulation. The controller identifies the next useful activity, while the planner handles movement toward its current destination. Dynamic replanning closes the gap between a route computed at dispatch and the aisle conditions encountered during execution.

A* reduced expansions relative to uninformed search on the normal-cost queries. The increase from 81 to 504 expansions under rush-hour costs illustrates the importance of the heuristic and cost model. Manhattan distance measures minimum movement steps; it does not predict the full penalty of travelling through the central band.

Greedy search happened to match the best route costs in both sets of store queries. Its small expansion counts are attractive in this example, but it ignores accumulated cost. Additional maps with misleading detours and unequal-cost alternatives are needed before choosing it as the default for broader workloads.

DFS demonstrates a contrasting behavior: a valid route can be expensive when the search follows a long branch before reaching the goal. Its large aggregate route cost makes it useful as a teaching comparison, rather than evidence of a suitable retail navigation policy.

The disturbance experiment also clarifies what autonomy means here. The robot reacts to fully known changes and follows predetermined task priorities. It does not learn from experience, forecast demand, or infer hidden shopper intentions. Calling the controller autonomous describes its ability to continue selecting and executing tasks without individual dispatch commands.

A. Limitations and future work

Only one store layout and a small, fixed set of queries were evaluated. The autonomous trials use deterministic events rather than independent random samples. No confidence intervals, real-time latency guarantees, or physical-robot performance claims follow from these measurements. The obstacle schedule deliberately affects future route cells when possible, so it is a controlled stress test rather than a model of observed aisle traffic.

The environment is fully observable, with discrete movement and simplified stock and battery rules. Decorative shoppers do not provide a realistic collision model. The simulation does not include sensor noise, localization error, acceleration, braking distance, or communication delay. CSP and Minimax remain separate demonstrations, and a multi-robot coordination result has not been established.

Future evaluation should vary layouts, start-goal pairs, obstacle duration, and order arrival rates. Repeated seeded scenarios could compare completion time, waiting time, failed tasks, and energy use. A priority queue could improve frontier management. A fleet extension would need actual shared-space constraints and collision checks before integrating the workbench assignment ideas.

VI. CONCLUSION

Retail Run provides an interactive demonstration of search-based navigation, task automation, and recovery from changing aisle conditions. A* and Greedy achieved low expansion counts on the selected normal-cost queries, while DFS produced substantially longer routes. Temporary obstacles increased travel cost in the autonomous test while preserving 11 mission completions. The results support the behavior of this implementation on its tested scenarios and provide a basis for broader evaluation, rather than establishing general optimality or readiness for a physical store.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020. [Online]. Available: https://aima.cs.berkeley.edu/.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100-107, 1968. doi: 10.1109/TSSC.1968.300136.

[3] S. Mulik, “Smart Retail Store Automation,” FA1 project report, Pimpri Chinchwad College of Engineering, unpublished coursework.

[4] M. Vadapure, S. Suryawanshi, and S. Kolhe, “Rescue Run,” project report, unpublished coursework.

[5] S. Mulik and R. Tere, “Retail Run,” browser application and evaluation artifacts, 2026. Implementation: engine.js, app.js, autonomy.js; measurements: report-work/measurements.json, recorded Oct. 27, 2026.